api_examples

October 22, 2026

1 Public API Examples

This notebook demonstrates the core `clif` package API: configuring an algebra,constructing multivectors, and computing inverses.

[1]:
```
from clif.context import Cl, Layout
from clif.multivector import Accum
from clif.inverse.fls import fls_inverse
from clif.inverse.euclidean import euclidean_inverse
```

1.1 Configuring an algebra

`Cl(p, q, r)` sets the global algebra to one with `p` positive, `q` negative, and `r` null basis vectors. `Layout([...])` is equivalent but takes a per-axis sign list.

[2]:
```
Cl(6)          # Euclidean 6D: Cl(6, 0, 0)
print("Euclidean 6D configured")

Layout([1, 1, 1, -1, -1, -1])   # mixed signature, still 6D
print("Mixed 6D configured")

Cl(6)          # back to Euclidean for the rest of the notebook
```

```
Euclidean 6D configured
Mixed 6D configured
```

1.2 Constructing multivectors

`Accum` holds a multivector as a NumPy array of 2^d real coefficients. Coefficients are indexed by ordinal: bit k set in the index means basis vector e_{k+1} is present in that blade.

[3]:
```
A = Accum.random()
print("Random 6D multivector (64 coefficients):")
print(A)
```

```
Random 6D multivector (64 coefficients):
00000000      -1.52597951
00000001       1.14575472
00000010       0.21538809
00000011      -0.29258761
```

00000100	-0.97132888
00000101	0.28606183
00000110	1.52511367
00000111	0.04189588
00001000	2.36402318
00001001	-0.17511576
00001010	1.12918338
00001011	-0.44848981
00001100	1.38451148
00001101	0.50999881
00001110	0.31443318
00001111	0.26355921
00010000	0.17202180
00010001	-0.76433139
00010010	0.92009273
00010011	0.78705338
00010100	0.15948751
00010101	-1.32390903
00010110	1.05406538
00010111	0.11366518
00011000	0.14775149
00011001	-0.10542735
00011010	1.48370179
00011011	-0.17662946
00011100	-1.08757436
00011101	-1.07046972
00011110	-0.53634074
00011111	0.23225844
00100000	1.40821701
00100001	-0.63203809
00100010	-1.19591295
00100011	0.51175933
00100100	-0.49200006
00100101	0.05440550
00100110	-1.24129105
00100111	0.58547814
00101000	-0.16696184
00101001	1.69365833
00101010	1.43048795
00101011	-0.40252987
00101100	0.09709135
00101101	-0.72930697
00101110	1.73295358
00101111	-1.11975876
00110000	1.46203320
00110001	0.53775587
00110010	0.06966411
00110011	-1.03156330

```
00110100      -0.38625815
00110101      -0.43266723
00110110      -0.88409630
00110111       0.26977824
00111000       0.88712220
00111001       2.22111766
00111010      -0.44734108
00111011      -0.73008167
00111100      -1.07631382
00111101       0.79130502
00111110       0.69732607
00111111      -1.09163064
```

[4]:
```python
# Scalar, vector, and pseudoscalar components
import clif.context as ctx

print(f"Scalar  (grade 0): A[0b000000] = {A.Reg[0]:.6f}")
print(f"e1      (grade 1): A[0b000001] = {A.Reg[1]:.6f}")
print(f"e1^e2   (grade 2): A[0b000011] = {A.Reg[3]:.6f}")
print(f"e1^..^e6 (grade 6, pseudoscalar): A[0b111111] = {A.Reg[63]:.6f}")
```

```
Scalar  (grade 0): A[0b000000] = -1.525980
e1      (grade 1): A[0b000001] = 1.145755
e1^e2   (grade 2): A[0b000011] = -0.292588
e1^..^e6 (grade 6, pseudoscalar): A[0b111111] = -1.091631
```

1.3 Computing inverses

1.3.1 FLS inverse (general multivector)

[5]:
```python
iA = fls_inverse(A)
check = A * iA
print("A * fls_inverse(A):")
print(check)
print(f"\nScalar component (should be 1.0): {check.Reg[0]:.15f}")
print(f"Max off-scalar residual:           {max(abs(check.Reg[1:])):.2e}")
```

```
A * fls_inverse(A):
00000000       1.00000000


Scalar component (should be 1.0): 1.000000000000001
Max off-scalar residual:           4.40e-14
```

1.3.2 Euclidean inverse (Euclidean signature, dimensions 6–13)

[6]:
```
iA2 = euclidean_inverse(A)
check2 = A * iA2
print(f"Scalar component: {check2.Reg[0]:.15f}")
print(f"Max off-scalar:   {max(abs(check2.Reg[1:])):.2e}")
```

```
Scalar component: 0.999999999999998
Max off-scalar:   5.00e-16
```

1.4 Algebra arithmetic

`Accum` supports `*` (geometric product), `+`, `-`, and scalar multiplication.

[7]:
```
B = Accum.random()

# geometric product is non-commutative in general
AB = A * B
BA = B * A

commutator = AB - BA
print(f"||AB - BA||_max = {max(abs(commutator.Reg)):.6f}  (non-zero means␣
  ↪non-commutative)")
```

```
||AB - BA||_max = 22.763992  (non-zero means non-commutative)
```

1.5 Verify across dimensions

[8]:
```
import numpy as np

for d in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]:
    Cl(d)
    A = Accum.random()
    iA = A.inverse()
    residual = max(abs((A * iA).Reg[1:]))
    print(f"Cl({d:2d}):  max residual = {residual:.2e}")
```

```
Cl( 1):  max residual = 0.00e+00
Cl( 2):  max residual = 1.11e-16
Cl( 3):  max residual = 1.11e-16
Cl( 4):  max residual = 9.71e-17
Cl( 5):  max residual = 1.67e-16
Cl( 6):  max residual = 2.58e-15
Cl( 7):  max residual = 6.45e-16
Cl( 8):  max residual = 3.22e-14
Cl( 9):  max residual = 1.00e-14
Cl(10):  max residual = 2.18e-15
Cl(11):  max residual = 3.23e-15
Cl(12):  max residual = 4.94e-15
```

```
Cl(13):  max residual = 4.38e-15
Cl(14):  max residual = 2.42e-07
Cl(15):  max residual = 6.12e-07
```